Phase 1 Static FJSP Mathematical Model

FJSP-AGV-RL Research Platform

June 7, 2026

1 Scope

This document describes the Phase 1 static flexible job shop scheduling problem (FJSP) model implemented by the repository. Transportation time, AGV routing, dynamic job arrivals, reinforcement learning, and graph neural network components are outside the Phase 1 scope.

2 Sets and Indices

Symbol	Meaning
J	set of jobs, indexed by j
O_j	ordered set of operations for job j, indexed by o
M	set of machines, indexed by m
A_{jo}	eligible machines for operation (j, o)

Operation (j, o) must precede $(j, o + 1)$ for every job j and operation position o where $o + 1 \in O_j$.

3 Parameters

Symbol	Meaning
p_{jom}	processing time of operation (j, o) on machine m
H	conservative scheduling horizon

The implementation uses a conservative horizon

$$H = \sum_{j \in J} \sum_{o \in O_j} \max_{m \in A_{jo}} p_{jom}.$$

This upper bound is not necessarily tight, but it is safe for the small CP-SAT baseline model.

4 Decision Variables

Here a and b denote operation identifiers such as (j, o). The explicit y variables are useful for a classical mixed-integer programming statement. The current OR-Tools implementation uses optional interval variables and `NoOverlap` instead of manually creating all y variables.

Symbol	Meaning
$x_{jom} \in \{0,1\}$	1 if operation (j,o) is assigned to machine m
$S_{jo} \geq 0$	start time of operation (j,o)
$C_{jo} \geq 0$	completion time of operation (j,o)
$C_{\max} \geq 0$	makespan
$y_{abm} \in \{0,1\}$	order variable for two operations a, b on machine m

5 Objective

The objective is to minimize the makespan:

$$\min C_{\max}.$$

6 Constraints

6.1 Machine Assignment

Each operation must be assigned to exactly one eligible machine:

$$\sum_{m \in A_{jo}} x_{jom} = 1, \qquad \forall j \in J,\ o \in O_j.$$

Machines outside A_{jo} are not valid choices and are never created as candidate assignments in the implementation.

6.2 Processing Time Link

If operation (j,o) is assigned to machine m, then its completion time must match its start time plus the selected processing time:

$$C_{jo} \geq S_{jo} + p_{jom} - H(1 - x_{jom}), \qquad \forall j, o,\ m \in A_{jo}.$$

6.3 Job Precedence

Operations within the same job must follow the given route order:

$$C_{jo} \leq S_{j,o+1}, \qquad \forall j \in J,\ o, o+1 \in O_j.$$

6.4 Machine Capacity

A machine can process at most one operation at a time. For any two distinct operations $a = (j,o)$ and $b = (k,q)$ that are both eligible for machine m, the disjunctive ordering is:

$$S_a + p_{am} \leq S_b + H(1 - y_{abm}) + H(2 - x_{am} - x_{bm}),$$

$$S_b + p_{bm} \leq S_a + H y_{abm} + H(2 - x_{am} - x_{bm}).$$

These constraints are active only when both operations are assigned to machine m. In the CP-SAT implementation, this logic is represented by optional intervals and a per-machine `NoOverlap` constraint.

6.5 Makespan Definition

The makespan must be no smaller than every final operation completion time:

$$C_{\max} \geq C_{j,last(j)}, \qquad \forall j \in J.$$

7 Feasibility Checker Mapping

The repository's feasibility checker validates the following model conditions:

- every operation appears exactly once;
- all job and operation ids are inside the instance range;
- every selected machine is eligible for the operation;
- recorded processing time matches the instance data;
- job precedence constraints are satisfied;
- no two operations overlap on the same machine;
- stored makespan equals the maximum recorded completion time.

8 Dispatching Rules

The strict dispatching rules use a common candidate tuple

$$(j, o, m, r_{jo}, s_{jom}, f_{jom}, p_{jom}),$$

where r_{jo} is the job ready time, s_{jom} is the earliest feasible start time on machine m, and $f_{jom} = s_{jom} + p_{jom}$ is the tentative finish time.

Rule	Operation selection	Machine selection
FIFO	smallest (r_{jo}, j, o)	earliest finish
SPT	smallest p_{jom} pair	selected with pair
EFT	smallest f_{jom} pair	selected with pair
MWKR	largest remaining route work	earliest finish
MOPNR	largest remaining operation count	earliest finish
Random	random ready pair	selected with pair

For MWKR, remaining route work is estimated as

$$W_{jo} = \sum_{q=o}^{last(j)} \min_{m \in A_{jq}} p_{jqm}.$$

For MOPNR, the ready job priority is its number of remaining operations:

$$N_{jo} = last(j) - o + 1.$$

9 Experiment Result Schema

Phase 1 experiment scripts write normalized CSV files under `experiments/results/`. Each row has the following fields:

The CSV layer is intentionally simple. It records what happened in each experiment without changing solver behavior or feasibility definitions. All Phase 1 experiment scripts use shared runner helpers that perform timing, call the solver, run the independent feasibility checker, collect failures, and copy optional solver metadata into the CSV record.

Field	Meaning
`experiment_id`	stable experiment identifier
`instance_name`	benchmark or toy instance name
`algorithm`	solver or dispatching rule name
`makespan`	objective value, empty if no schedule was returned
`feasible`	`PASS` or `FAIL` after feasibility checking
`runtime_s`	measured wall-clock runtime in seconds
`note`	solver status, error, or validation note
`objective_bound`	CP-SAT best objective bound, when available
`gap`	relative gap $(C_{\max} - LB)/\max(1, C_{\max})$, when available
`solver_wall_time`	CP-SAT internal wall time, when available

9.1 Batch Benchmark Entry Point

The batch experiment recursively discovers `*.fjs` files in stable path order. Each loaded static FJSP instance is evaluated with the selected Phase 1 heuristics and, by default, the CP-SAT reference solver. A configurable per-instance time limit bounds CP-SAT work on larger collections. Loader failures are retained as `LOAD` rows with `FAIL` status so that a malformed file cannot disappear silently from an experiment report.

9.2 Tracked FJSPLib Subset

The repository tracks a small real-data subset copied from the official ScheduleOpt FJSPLib repository under its CC-BY-SA-4.0 license: `https://scheduleopt.github.io/benchmarks/fjsplib`. The subset spans six source families and increasing instance sizes.

The local CP-SAT values above were independently validated by the feasibility checker. The FJSPLib BKS column is upstream reference data, not a value inferred by this repository. For `mk02`, `mk03`, `mk04`, `mk05`, `mk06`, `mk07`, `mk08`, `mk09`, `mt10c1`, `01a`, and `kacem4`, CP-SAT reports `FEASIBLE` under the 10-second limit. The batch therefore records time-limited feasible reference schedules, not locally proven optima.

The official benchmark table lists `kacem1` as 4×6, while the official old-format file begins with the header `4 5 5`. The repository reports the loadable file content as 4×5 and preserves the upstream BKS of 11.

The official benchmark table lists `fattahi6` as 3×2, while the official old-format file begins with `3 3 1.6`. The repository reports the loadable file content as 3×3 and preserves the upstream BKS of 320.

10 CP-SAT Diagnostics

For CP-SAT results, the implementation stores three optional diagnostics in the common `ScheduleResult` object:

$$LB = \text{BestObjectiveBound}, \qquad gap = \frac{C_{\max} - LB}{\max(1, C_{\max})},$$

and the solver-reported wall time. For an optimal solution on the toy instance, the bound equals the makespan and the reported gap is zero. Heuristic results leave these fields empty.

10.1 CP-SAT Time-Limit Sweep

The tracked `mk03` sweep fixes the CP-SAT worker count to one and varies only the time limit. Fixing the worker count reduces parallel-search noise and makes the comparison easier to inter-

Instance	Jobs	Machines	FJSPLib BKS	Local CP-SAT
fattahi1	2	2	66	66
fattahi2	2	2	107	107
fattahi3	3	2	221	221
fattahi4	3	2	355	355
fattahi5	3	2	119	119
fattahi6	3	3	320	320
fattahi7	3	5	397	397
fattahi8	3	4	253	253
mt10c1	10	11	927	927
mt10cc	10	12	908	908
mt10x	10	11	918	918
01a	10	5	2505	2586
02a	10	5	2228	2534
03a	10	5	2228	2602
04a	10	5	2503	2591
kacem1	4	5	11	11
kacem2	10	7	11	11
kacem3	10	10	7	7
kacem4	15	10	11	15
Hurink edata ft06	6	6	55	55
Brandimarte mk01	10	6	40	40
Brandimarte mk02	10	6	26	28
Brandimarte mk03	15	8	204	220
Brandimarte mk04	15	8	60	62
Brandimarte mk05	15	4	172	180
Brandimarte mk06	10	15	57	74
Brandimarte mk07	20	5	139	159
Brandimarte mk08	20	10	523	523
Brandimarte mk09	20	10	307	340
Brandimarte mk10	20	15	193	302

pret.

Limit (s)	Makespan	Status	Bound	Gap
1	417	FEASIBLE	20	0.952038
3	285	FEASIBLE	20	0.929825
10	220	FEASIBLE	20	0.909091

Longer limits improve the feasible schedule in this run, but no row proves optimality. The default benchmark batch uses the solver's default worker configuration and is recorded separately. Worker count is part of the experimental configuration and must not be hidden during comparisons.

10.2 CP-SAT Worker-Count Sweep

The tracked mk03 worker sweep fixes the time limit to 10 seconds and varies the CP-SAT worker configuration. A worker count of auto leaves the solver's worker choice at its default value.

Every row passed the independent feasibility checker. The table is one timed observation, not a stable ranking of worker configurations: parallel CP-SAT search can vary between runs.

Workers	Makespan	Status	Bound	Gap
1	220	FEASIBLE	20	0.909091
2	207	FEASIBLE	20	0.903382
4	207	FEASIBLE	20	0.903382
auto	209	FEASIBLE	75	0.641148

In this observation, the automatic configuration produced the strongest bound, while two and four workers produced slightly better incumbents.

10.3 Repeated CP-SAT Worker Observations

The repeated experiment keeps the same mk03 instance and 10-second limit, then runs each worker configuration three times. It stores every raw feasibility-checked run and a separate aggregate CSV. The reported standard deviation is the population standard deviation of the observed makespans.

Workers	Pass	Best	Mean	Std. dev.	Best bound	Best gap
1	3/3	220	220.000000	0.000000	20	0.909091
2	3/3	204	204.000000	0.000000	20	0.901961
4	3/3	204	204.000000	0.000000	21	0.897059
auto	3/3	204	204.000000	0.000000	88	0.568627

All 12 schedules passed the independent feasibility checker. Two, four, and automatic workers consistently reached the upstream BKS of 204 in this small sample. Because CP-SAT still reported FEASIBLE, these schedules remain time-limited feasible references rather than locally proven optima.

10.4 Repeated CP-SAT Benchmark Matrix

The controlled matrix recursively scans the tracked static FJSPLib subset. Every instance is evaluated twice with one worker and twice with the automatic worker configuration under a 3-second limit. Raw rows and per-instance aggregates are stored separately.

Workers	Instances	Feasible rows	Optimal rows	Mean best	Mean gap
1	30	30	12	674.333	0.451173
auto	30	30	12	590.367	0.273244

All 120 schedules passed the independent feasibility checker. A dedicated report builder, exp_013_cpsat_report.py, converts the aggregate CSV into a Markdown report with each instance's best worker row, objective bound, gap, and status. The report keeps the distinction between valid FEASIBLE schedules and locally proved OPTIMAL schedules explicit.

10.5 Heuristic Benchmark Matrix

The heuristic matrix evaluates every deterministic Phase 1 rule once on each tracked FJSPLib instance. The two random families use fixed seeds 42, 123, and 999. Their ranges therefore describe reproducible observations rather than an uncontrolled stochastic sample.

All 420 schedules passed the independent feasibility checker. No single rule dominates every instance: MWKR is strongest on mk01, mk02, mk06, mk10, mt10cc, fattahi1, kacem2, and kacem4; FIFO is strongest on mk04, mk05, mk08, mk09, fattahi5, 02a, 03a, ft06, and

Instance	Best deterministic rule	Random range	DispatchRandom range
mk01	DispatchMWKR (44)	148–165	92–125
mk02	DispatchMWKR (30)	134–169	86–113
mk03	DispatchFIFO/MWKR (204)	1086–1206	488–563
mk04	DispatchFIFO (77)	279–292	143–177
mk05	DispatchFIFO (185)	541–584	287–323
mk06	DispatchMWKR (67)	594–673	222–238
mk07	DispatchMOPNR (160)	689–854	345–427
mk08	DispatchFIFO (531)	2306–2560	755–916
mk09	DispatchFIFO (341)	2071–2200	817–960
mk10	DispatchMWKR (243)	1973–2087	610–710
mt10c1	DispatchMOPNR (1225)	3348–3394	1446–1728
mt10cc	DispatchMWKR (1121)	3348–3394	1597–1841
mt10x	EarliestFinishTime/DispatchEFT (1173)	3095–3394	1744–1849
01a	DispatchFIFO (3056)	10212–10398	4132–4525
02a	DispatchFIFO (2443)	10010–10305	4401–4804
03a	DispatchFIFO (2413)	10025–10207	4197–4898
04a	DispatchMOPNR (3070)	10177–10380	4090–4495
fattahi1	DispatchMWKR (66)	86–135	86–135
fattahi2	DispatchFIFO/MWKR (107)	107–192	107–149
fattahi3	DispatchFIFO/MOPNR (221)	298–442	221–361
fattahi4	DispatchFIFO/MWKR/MOPNR (367)	409–699	407–626
fattahi5	DispatchFIFO (119)	207–246	150–220
fattahi6	EarliestFinishTime/DispatchEFT (330)	557–707	470–557
fattahi7	multiple ties (397)	491–595	462–598
fattahi8	EarliestFinishTime/DispatchEFT (273)	275–317	341–450
ft06	DispatchFIFO (55)	146–152	71–112
kacem1	DispatchFIFO (11)	27–82	26–44
kacem2	DispatchMWKR (12)	62–102	50–88
kacem3	DispatchFIFO/MWKR/MOPNR (8)	42–77	50–72
kacem4	DispatchMWKR (12)	114–159	71–138

kacem1, ties MWKR on mk03 and fattahi2, and ties MOPNR on fattahi3 and fattahi4. EarliestFinishTime ties DispatchEFT on fattahi6, mt10x, and fattahi8. MOPNR is strongest on mk07, 04a, and mt10c1; FIFO is strongest on 01a; and FIFO, MWKR, and MOPNR tie on kacem3. Ready-operation random dispatching is consistently better than the simple job-order random baseline in this tracked subset, but remains seed-sensitive.

10.6 BKS-Relative Heuristic Quality

The quality report reads structured upstream references from `instances/fjsplib_bks.csv`. For heuristic family h and instance i, it computes

$$gap_{\mathrm{BKS}}(h,i) = \frac{C_{\max}(h,i) - \mathrm{BKS}_i}{\mathrm{BKS}_i}.$$

The upstream BKS is a comparison reference, not a local optimality proof.

Algorithm family	Mean best gap	Maximum best gap
FIFO	4.862683	25.090909
SPT	2.291656	6.088083
EarliestFinishTime	0.392423	1.384615
DispatchFIFO	0.151281	0.370370
DispatchSPT	1.393257	4.564767
DispatchEFT	0.405199	1.384615
DispatchMWKR	0.158760	0.364924
DispatchMOPNR	0.193880	0.466667
Random	3.141676	9.421053
DispatchRandom	1.318093	6.142857

DispatchFIFO has the lowest mean best gap on the current thirty-instance subset. The sample remains deliberately small, so broader benchmark coverage should precede additional dispatching rules.

10.7 Tracked Benchmark Metadata Audit

Before expanding the benchmark subset, the metadata audit recursively discovers tracked FJSPLib files, matches them one-to-one with `instances/fjsplib_bks.csv`, and loads every discovered file. A row passes only if the tracked file exists, its upstream reference row exists, and the loader accepts the instance. All thirty currently tracked rows pass. This gate prevents silently omitted files and stale metadata from entering quality comparisons.

11 Conclusion and Limitations

Phase 1 is now a reproducible static FJSP baseline rather than an optimization claim for every benchmark. The strongest current properties are the independent feasibility checker, audited benchmark provenance, repeated CP-SAT diagnostics, and BKS-aware heuristic summaries. The main limitations are deliberate: the benchmark subset is still moderate, CP-SAT runs are time-limited on larger instances, and no AGV, RL, GNN, dynamic rescheduling, or full paper reproduction is included in this phase. These boundaries keep the codebase maintainable and make future Phase 2 extensions easier to review.

Threat	Current mitigation
Time-limited CP-SAT may return `FEASIBLE` schedules without proof	Reports keep status, bound, solver gap, BKS gap, and local-proof fields separate.
Benchmark subset is moderate	Every tracked file has provenance, LB/UB metadata, and loader audit coverage.
Random heuristics are seed-sensitive	Random families use fixed seeds and report ranges instead of hidden single samples.
Upstream BKS is external reference data	Reports distinguish reaching UB from locally proving `OPTIMAL`.
Phase 2 topics could blur scope	AGV, RL, GNN, and dynamic rescheduling remain outside Phase 1.